

Адаптеры над итераторами – это обёртки над итераторами, которые изменяют поведение итераторов. Они упрощают работу с контейнерами и бывают очень полезны в стандартных алгоритмах. Адаптеры находятся в заголовочном файле `<iterator>`

std::reverse_iterator

Это итератор, перебирающий элементы в обратном направлении. Обратный итератор базируется на двунаправленном итераторе (BidirectionalIterator). После инкрементирования он будет указывать на предыдущий элемент контейнера.

```
#include <iostream>
#include <list>

int main() {
    std::list<int> list { 1, 2, 3, 4, 5 };
    std::list<int>::iterator it = list.end();
    std::list<int>::reverse_iterator rit = std::make_reverse_iterator(it);
    while (rit != list.rend()) {
        std::cout << *rit << ' ';
        ++rit;
    }
}
```

Данное преобразование возможно только для тех итераторов, которые поддерживают декремент (--). То есть для нашего пользовательского итератора (см. лекцию 13) данное преобразование недопустимо.

std::move_iterator

Данный итератор поддерживает семантику перемещения элементов контейнера.

```
#include <iostream>
#include <set>
#include <vector>

int main() {
    std::set<std::string> words { "mango", "orange", "apple", "pear" };
    std::vector<std::string> newWords { "cucumber", "tomato", "pumpkin" };
    words.insert(std::make_move_iterator(newWords.begin()),
                std::make_move_iterator(newWords.end()));

    std::cout << " words: "; //Будут выведены все слова
    for (auto word : words) {
        std::cout << '\'' << word << "' ";
    }
    std::cout << "\nnewWords: "; //Будет '' '' ''
    for (auto word : newWords) {
        std::cout << '\'' << word << "' ";
    }
}
```

Ещё про move_iterator

```
int main() {  
  
    std::set<std::string> words { "mango", "orange", "apple", "pear" };  
    std::vector<std::string> newWords { "cucumber", "tomato", "pumpkin" };  
    std::vector<std::string>::iterator it = newWords.begin();  
  
    auto it_move = std::make_move_iterator(it);  
  
    words.insert(it_move, it_move + 2);  
    std::cout << " words: "; // 'apple' 'cucumber' 'mango' 'orange' 'pear' 'tomato'  
    for (auto word : words) {  
        std::cout << '\'' << word << "' ";  
    }  
    std::cout << "\nnewWords: "; // ' ' 'pumpkin'  
    for (auto word : newWords) {  
        std::cout << '\'' << word << "' ";  
    }  
}
```

Потоковые итераторы

Потоковые итераторы позволяют работать с потоком как с контейнером и использовать стандартные алгоритмы.

Итератор потока ввода `'std::istream_iterator'` предназначен для чтения данных из объекта `'std::basic_istream'` (например, из `'std::cin'` или файла, открытого на чтение). При создании итератора потока ввода требуется инстанцировать его типом считываемых элементов и передать в конструктор поток.

Считывание элементов происходит в момент инкрементирования итератора через оператор '++'. Операция инкрементирования использует операцию '>>' потока для чтения. Операция разыменования '*' возвращает только константную копию элемента

```
#include <iostream>
#include <iterator>

int main() {
    std::istream_iterator<int> input_iter(std::cin);
    std::ostream_iterator<int> output_iter(std::cout, " ");
    //В данном случае ввод будет продолжаться до тех пор, пока на вход поступают int
    while (input_iter != std::istream_iterator<int>()) {
        if (*input_iter % 2 == 0) {
            *output_iter = *input_iter;
        }
        input_iter++;
    }
}
```

Итератором, который обозначает конец последовательности, является итератор по умолчанию 'std::istream_iterator<int>()'. Это вызов конструктора по умолчанию std::istream_iterator<int>, который создаёт специальный итератор. Он не привязан ни к какому потоку и служит маркером конца для сравнения с рабочими итераторами. Разыменовывание приводит к неопределённому поведению.

Output iterator (итераторы вставки)

Итераторы вставки заменяют операцию присваивания на операцию вставки. Это может быть полезно при добавлении группы новых элементов в контейнер. Существует три вида итераторов вставки:

'std::front_insert_iterator<Container>' – использует метод 'push_front' для вставки элементов в начало контейнера;

'std::back_insert_iterator<Container>' – использует метод 'push_back' для вставки элементов в конец контейнера;

'std::insert_iterator<Container, Iter>' – вставляет элемент в произвольную позицию контейнера, заданную итератором Iter. Output итераторы принимают контейнер.

```
#include <iostream>
#include <vector>

int main() {
    int a[6] = {3,7,2,9,5,1};
    std::vector<int> vec;
    std::copy (a, a + 6, std::back_insert_iterator<std::vector<int>>(vec));
    for (const auto&x : vec) {
        std::cout << x << ' ';
    }
}
```

std::back_inserter

Это шаблонная функция, которая принимает контейнер (выводит его тип) и возвращает back_insert_iterator. Требуется наличия метода push_back у контейнера.

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <iterator>

int main() {
    int a[6] = {3, 7, 2, 9, 5, 1};
    std::vector<int> vec;
    std::copy(a, a + 6, std::back_inserter(vec));
    for (const auto& x : vec) {
        std::cout << x << ' ';
    }
}
```

Можно и без неё:

```
std::back_insert_iterator<std::vector<int>> rec_it(vec);
std::copy(a, a + 6, rec_it);
```


Пример

```
#include <array>
#include <iostream>
#include <iterator>
#include <string>

int main() {
    std::array<int, 3> a1{1, 2, 3};
    std::array<int, 3> a2 = {1, 2, 3};
    std::array<std::string, 2> a3 = {std::string("a"), "b"};

    std::sort(a1.begin(), a1.end());
    std::reverse_copy(a2.begin(), a2.end(),
                     std::ostream_iterator<int>(std::cout, " ")); // В консоли: 3 2 1
    std::cout << '\n';

    for (const auto& s : a3)
        std::cout << s << ' '; // В консоли: a b
}
```

`std::ostream_iterator<int>` — это адаптер, который позволяет использовать алгоритмы STL для вывода данных в поток

Ассоциативные контейнеры

- **Ассоциативные контейнеры** – это контейнеры, которые автоматически сортируют свои входные данные, когда эти входные данные вставляются в контейнер. По умолчанию ассоциативные контейнеры сравнивают элементы с помощью `operator<`.
- Ассоциативные контейнеры автоматически сортируют свои элементы **по некоторому критерию**. **Критерий** представляется в **виде функции**, которая сравнивает либо значения, либо специальные ключи, определяемые для этих значений.

std::map

Это упорядоченный ассоциативный массив ключ-значение. Для сортировки применяется КЧД. Среднее время работы вставки, удаления и поиска работает за $O(\log(n))$

```
#include <iostream>
#include <map>

int main() {
    std::map<char, int> f;
    f['a'] = 10;
    f['b'] = 30;

    std::map<char, int> s(f.begin(), f.end());
    std::map<char, int> t(s);

    for(const auto& element: f) {
        std::cout << element.first << ' ' << element.second << '\n';
    }

    std::cout << t['a'] << '\n';

    for(const auto& [el1, el2]: t) { //since C++17
        std::cout << el2 << '\n';
    }
}
```

Элементы std::map

Каждый элемент map фактически представляет объект типа **std::pair<const Key, Value>**, который хранит, как ключ, так и значение. С помощью полей first и second объекта можно получить, соответственно, ключ и значение элемента.

```
for(const auto& element: f) {  
    std::cout << element.first << ' ' << element.second << '\n';  
}
```

Итератор std::map представляет собой указатель на **pair<const Key, Value>**

Основные методы std::map

```
int main() {
    std::map<char, int> f { {'c', 20}, {'d', 80}, {'e', 60}};
    f['a'] = 10;
    f['b'] = 30;
    f.insert(std::pair('x', 52));
    f.insert(std::map<char, int>::value_type('c', 12));
    f.insert(std::make_pair('y', 44));
    f.insert({'z', 34});

    //insert возвращает pair <iterator, bool>
    std::cout << f.insert(std::map<char, int>::value_type('a', 24)).second << ' ';

    //at() возвращает значение value
    std::cout << f.at('b') << ' ';

    auto it = f.begin();
    //erase и find возвращают итераторы типа pair<const Key, Value>*
    std::cout << f.erase(++it)->first << ' '; //erase возвращает следующий за удалённым
    std::cout << f.find('a')->second << ' '; //find у map свой, ищет по ключу
}
```

operator[]

operator[] с несуществующим ключом всегда выполняет вставку элемента в контейнер значением по умолчанию:

```
std::map<int,int> m;  
std::cout << m[0] << ' ' << m[1] << ' ' << m[3]; // 0 0 0  
m[1] = 20; //при наличии такого ключа значение перезаписывается
```

operator[] не является константным:

```
template <typename T>  
void f(const T& t) {  
    std::cout << t[0];  
}  
  
int main() {  
    std::map<int,int> m;  
    m[0] = 0;  
    m[1] = 1;  
    f(m);  
}
```

'this' argument has type 'const std::map<int, int>', but method is not marked const
mapped_type& operator[](key_type&& __k);

insert vs operator []

insert: когда нужно добавить элемент, но не обновлять существующий, и важно узнать, произошла ли вставка на самом деле.

operator[]: когда нужно либо обновить существующий, либо создать новый (и неважно, существовало ли значение).

lower_bound(key), upper_bound(key), find(key)

```
std::map<char, int> f { {'c', 20}, {'d', 80}, {'e', 60}};  
f['a'] = 10;  
f['b'] = 30;  
  
std::cout << f.lower_bound('b')->first << ' ' << ' '; //>='b'  
std::cout << f.upper_bound('b')->first << ' ' << ' '; //>'b'  
  
std::cout << f.find('c')->second << ' ' << ' '; //20  
std::cout << f.find('r')->second << ' ' << ' '; //возвращает end(), разыменовываем, UB
```

Если не найдены соответствующие позиции, результатом будет указатель на end(),
разыменовывая который получим UB

Можно использовать свой компаратор (о классах сравнения и лямбда потом)

```
bool fcomp (char x, char y) {  
    return x > y;  
}  
  
int main() {  
    bool (*fn_pt)(char, char) = fcomp;  
    std::map<char, int, bool (*)(char, char)> f(fn_pt);  
  
    f['a'] = 10;  
    f['b'] = 30;  
  
    for(const auto& x: f) {  
        std::cout << x.first << ' ' << x.second << ' ' << '\n';  
    }  
}
```

Обратите внимание, что нужен именно указатель на функцию, потому что шаблонный аргумент должен быть типом

Если использовать свой класс в качестве ключа

```
struct myTest{
    int a = 0;
    int b = 3;
    myTest(int a, int b):a(a),b(b) {}
    bool operator < (const myTest& other) const {
        return (a < other.a) || (b < other.b);
    }
};

std::ostream& operator << (std::ostream& out, const myTest& my) {
    return out << my.a << ' ' << my.b << ' ';
}

int main() {
    myTest A(3,5);
    std::map<myTest, char> my;
    my[A] = 'a';
    my[{4,6}] = 'b';
    my[{2,2}] = 'b';
    for(const auto& x: my) {
        std::cout << x.first << ' ' << x.second << '\n';
    }
}
```

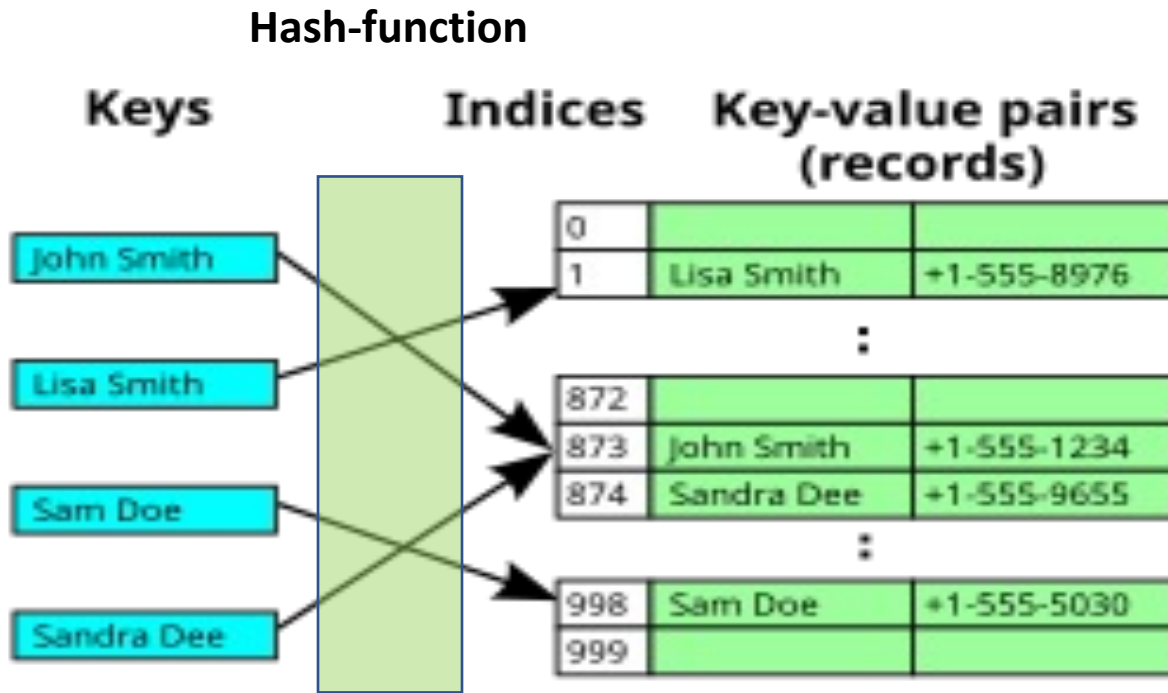
std::set

Это контейнер, в котором хранятся **уникальные элементы**, дублирование которых запрещено. **Элементы отсортированы по их значениям.** Принцип работы такой же, как и в map. Вместо пары ключ-значение в set хранятся только константные ключи. В set нет оператора [].

По сути, set – это не массив, а сбалансированное бинарное дерево, то есть там хранится вершина и указатели на потомков.

std::unordered_map.

Это такой же ассоциативный массив из ключа в значение, но среднее время работы вставки, удаления и поиска работает почти за $O(1)$. Для организации данной структуры применяется Hash-таблица.



Hash-таблица – это структура данных, реализующая ассоциативный массив, её устройство представляет собой двухэтапный процесс.

На первом этапе к ключу применяется [hash-функция](#). Это функция, которая на вход принимает набор байт (ключ), а на выходе обычно даёт какое-то число. Функция используется максимально сложная, чтобы максимально избежать повторений. Это число на втором этапе используется для поиска значения.

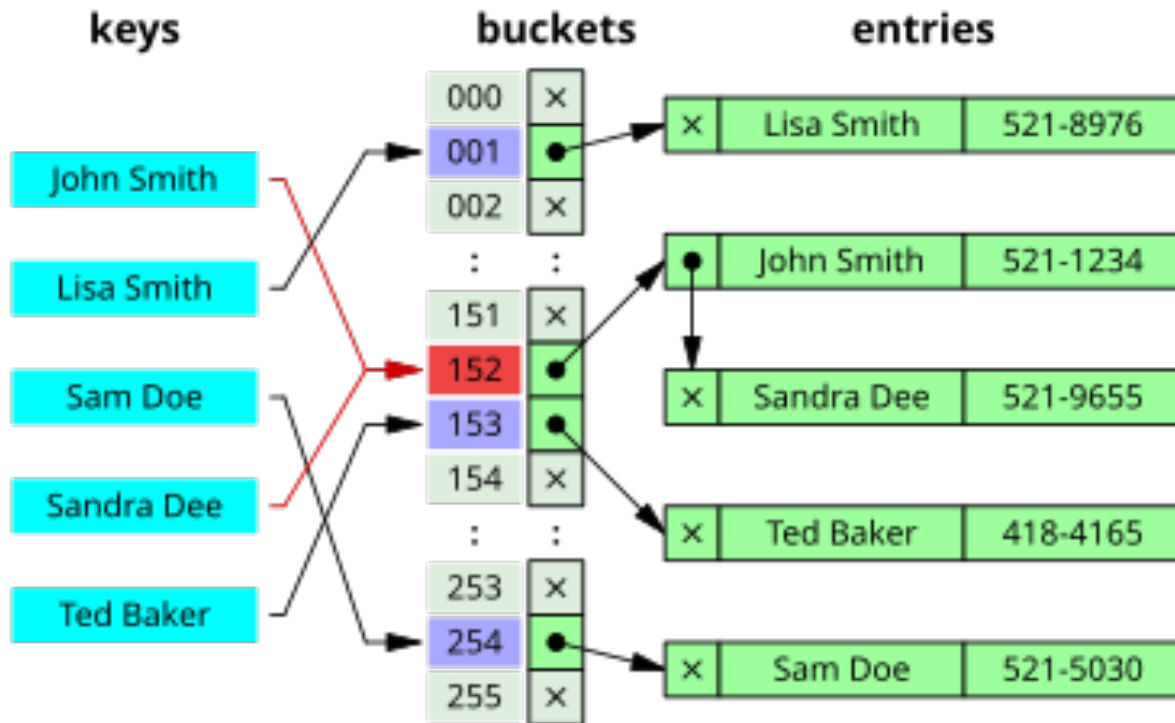
```
template<
    typename Key,
    typename T,
    typename Hash = std::hash<Key>,
    typename KeyEqual = std::equal_to<Key>,
    typename Allocator = std::allocator<std::pair<const Key, T>>
> class unordered_map;
```

Применение hash-таблицы даёт ооколоконстантное время операций за счёт того, что каждый элемент хранится по своему уникальному индексу, вычисляемому через hash-функцию. При вычислении индекса значение hash-функции масштабируется на размер таблицы, поэтому он (размер) не должен меняться при выполнении операций. Совпадение индексов называется *коллизией*. Есть несколько стратегий обработки коллизий. В `std::unordered_map` используется *метод списков*.

Главное преимущество использования hash-таблицы – очень короткое время доступа. Конфликты иногда могут возникать, но шансы практически равны нулю, если выбрать очень хорошую hash –функцию. Если в качестве ключа используется свой класс, необходимо определять для него свою hash-функцию.

Метод списков

В каждой вершине хранится связанный список или вектор, но чаще `forward_list`. Коллизии просто приводят к тому, что появляются списки длиной более одного элемента.



Элементы с одинаковым хэшем кладутся в соответствующий этому хэшу сегмент (*bucket*). При возрастании числа коллизий скорость операций с таблицей падает, поэтому необходимо производить рехэширование (*rehashing*) - увеличивать число сегментов и пересчитывать все хэши элементов, соответственно, перевыделять память для таблицы и вставлять их заново. Это дорогостоящая по времени и памяти операция, поэтому приходится искать компромисс между числом коллизий и частотой рехэширования.

Ещё контейнеры:

- multiset – это set, в котором **разрешены повторяющиеся элементы**.
- multimap (также называемый словарем) – это map, который позволяет **дублировать ключи**. Реальные словари – это multimap'ы: ключ – это слово, а значение – это значение слова. Все ключи отсортированы в порядке возрастания, и вы можете искать значение по ключу. Некоторые слова могут иметь несколько значений, поэтому словарь – это multimap, а не map.
- Ещё существуют неупорядоченные ассоциативные контейнеры, о которых не говорили, но принцип работы схож: unordered set, unordered multiset, и unordered multimap.